

Failure modes

- I tested simultaneous accepts, accept/cancel and competing reservations across processes. Balance checks and writes share SQLite's lock, so commands cannot spend the same funds. Receipts handle retries; a fresh command ID cannot fill the request twice.
- A failed response leaves payment uncertain. I tested process exits after debits, the first leg write and around commit. Before commit, everything rolls back; after it, receipt replay returns the result. Interrupted VOID refunds roll back both payments.
- The quote cap could exclude better offers. Replacement requires a better payout without shorter expiry; refund and reservation commit together. Replacements cannot be withdrawn. Acceptance rechecks the selected offer; expiry closes it without choosing another maker.
- Cleanup must not decide whether an expired quote is usable. Time is checked under the lock; backwards time is rejected. Recovery rotates work so request churn cannot starve payouts. Late finalization preserves uncontested results; disputes cannot extend fallback. Settlement waits for every leg and pays only once.
- Conserved totals can still hide a refund to the wrong person. Audits check recipients and backing. Validation rejects lossy amounts and altered terms; escaped asynchronous work cannot write.

Separate makers per leg remain design only. Committing leg 1 early or forgetting rivals could strand money. I would keep provisional matches in one transaction: if leg 2 of 3 cannot fill, reject everything and release every reservation. Whole-ticket acceptance already rejects invalid legs together and rolls back partial writes.